

Codebase Map Visualization – Improvement Recommendations

Overview and Key Challenges

The current **codebase graph** visualizer is powerful but struggles with **scale and clarity**. With ~2k nodes and ~5k edges, the graph becomes an unreadable "hairball" and lags under the rendering and physics load. The core issue is **edge density, not just node count** ¹ – too many connections create visual spaghetti and slow down the simulation. To improve both performance and usability, the solution is to adopt **multi-scale aggregation with progressive disclosure** ². In practice, this means *clustering nodes into higher-level groups by default*, **hiding or bundling less important edges**, and revealing detail on demand rather than showing everything at once.

Below we outline recommendations across the pipeline – from data scraping to visualization – to achieve a more scalable, interactive, and comprehensible code map.

1. Data Extraction & Graph Schema Enhancements

Improvements should begin at the **scraping stage**. Enriching the graph data with more metadata and hierarchy will enable smarter visualization choices later ³:

- **Add Node Metadata:** Extend each node with fields like `kind` (e.g. *package, module, file, class, function, component, hook, etc.*) and an **architectural layer tag** (e.g. *ui, api, domain, infra, tests*) ⁴. Additional properties like **size/LOC**, code **churn** or **"hotness"** (recent commit count) and last modified timestamp can indicate complexity or volatility ⁵. Mark **entry points** (such as main files, root components, API handlers) with a flag for emphasis ⁶. These metadata fields let the viewer filter or style nodes by importance, layer, or stability.
- **Hierarchical Grouping:** Instead of a flat list of nodes, organize them into a **deterministic hierarchy**. For example, define groups for *package* → *module (folder)* → *file* → *symbol*. Each node should carry a `cluster_id` corresponding to the group it belongs to, and possibly a path of parent cluster IDs up the hierarchy ⁷ ⁸. This can be done by analyzing file paths or namespaces during the scrape. The scraper can output separate JSONs for each Level of Detail (LOD) – e.g., a **LOD0 graph** of only top-level clusters, LOD1 for modules, LOD2 for files, etc ⁸ – or embed cluster info in the main JSON. These **hierarchy IDs** will allow the front-end to collapse/expand clusters easily.
- **Edge Semantics and Weight:** Enrich edges with a **semantic type** (e.g. *import, function-call, usage, inherits, tests, etc.*) rather than generic labels ⁹. The current extraction already distinguishes some relationships (e.g. `"page_uses_component"`, `"calls_api"`) – this should be standardized into broader categories (import vs. call vs. data flow, etc.). Also compute a **weight** or **strength** for

edges, such as the number of call sites or references behind that connection ¹⁰. For instance, if function A calls function B in 10 places, the A→B edge weight = 10. This helps decide which edges are significant. Additionally, tag edges with a **distance class** (local vs. cross-module vs. cross-package) ¹⁰. This metadata will support filtering (e.g. show only inter-module dependencies at high level) and bundling of multiple similar connections.

- **Git & Ownership Data:** Leverage the `extract_git_metadata.py` script (and related tooling) to attach “hotness” metrics (e.g., commits in last 30 days, number of authors) to nodes ⁵. This can highlight hotspots in the visualization (e.g. coloring nodes by churn). If possible, tag files with **owner/team** or *project module* (even via folder naming conventions) ¹¹ – giving each node an “owner” or domain grouping that could translate to color or cluster in the graph.

By baking in this richer schema at build time, the viewer can make informed decisions about what to display at various zoom levels ³. In short, **scrape more context** (structure, importance, semantics) so the front-end doesn’t have to guess how to simplify the graph.

2. Multi-Scale Clustering and Layered View

To prevent visual overload, implement a **multi-scale graph view**. The graph should be navigable like a map – start with an overview (few aggregate nodes) and zoom/drill down to reveal detail ¹². Key strategies:

- **Hierarchical Clustering (LOD):** Treat clusters (grouped nodes) as *first-class nodes*. At the highest level, collapse the entire codebase into broad clusters (e.g. each top-level package or feature area appears as one node). At intermediate zooms, clusters represent modules or directories, and at deepest zoom, individual files or functions ⁸ ¹³. For example:
- **LOD0 (System Overview):** show one node per major subsystem or package (perhaps derived from the top folder names or domain areas). Only show edges that represent *cross-package dependencies*, greatly reducing edge count.
- **LOD1 (Module Level):** when zooming in or expanding, each cluster node breaks into its child nodes (e.g. files or sub-modules within a package). Edges now show *interactions between modules* in that subsystem ⁸.
- **LOD2 (File/Class Level):** deeper zoom reveals individual files/classes and their connections within the module.
- **LOD3 (Function/Detail Level):** only when focusing on a specific file or component do we show the full symbol-level graph (functions, hooks, etc.). This is your current detailed graph, but it would be confined to a limited region of the view (or a separate detail pane) rather than everything at once.

Each level of detail should have its own aggregated graph data (which can be precomputed by the scraper or derived on the fly). Notably, **cluster nodes** should carry summarized info about what they contain (count of children, sum of sizes, etc.) so they can be visualized proportionally ¹⁴. The front-end should default to a high-level LOD for initial render, so the user sees an **organized overview instead of thousands of individual nodes** ¹³.

- **Progressive Disclosure on Zoom/Click:** Integrate the zooming UI with LOD switching. For example, as the user zooms closer or double-clicks a cluster node, replace that cluster node with its constituents (and optionally fade in the new edges) ¹⁵. Conversely, zooming out or collapsing should replace detail nodes with their parent cluster node. This **drill-down interaction** lets the user

explore the graph stepwise, preventing the entire graph from ever being fully expanded unless explicitly needed. It might be useful to implement this via a stateful approach – e.g., a **“graph patching”** where expanding a node merges its children nodes and edges into the current view ¹⁶ ¹⁷. This avoids reloading an entire new dataset for each zoom change and supports *partial expansion*: you could be viewing most of the system at a coarse level, but a specific area in fine detail ¹⁶ ¹⁷.

- **Edge Bundling and Aggregation:** When nodes are collapsed into clusters, also **bundle the edges** between clusters. Instead of rendering dozens of individual edges between nodes of cluster A and cluster B, render a single **“super-edge”** between cluster A and B that represents the collective connections ¹⁸. This super-edge could have a thickness or weight label indicating how many actual links it encapsulates ¹⁸. On-demand, these can split out: e.g., on hover or when expanded, you temporarily show the individual edges. Bundling drastically reduces clutter at high levels – edges become fewer and more meaningful. Techniques like **curved or spline edges** can differentiate intra-cluster vs. inter-cluster links and improve readability when drawing aggregated connections ¹⁹.
- **Filtering by Importance (Backbone View):** Introduce modes to view only the most significant relationships. For instance, a **“backbone” mode** might show only nodes above a certain centrality or importance threshold (like PageRank or degree) and only the strongest edges ²⁰. This extracts the **core architecture** from the graph. Another approach: allow a *spanning tree* or flow from a given entry-point (e.g., simulate a request path through the graph) to answer specific “how does data move from A to B” questions ²¹. These filtered views further reduce complexity for certain use cases.

By implementing multi-scale clustering and smart edge bundling, **the default view will be far less dense**, improving performance (fewer elements to simulate and draw) and usability (the high-level structure becomes apparent). As the project plan succinctly states: *“collapse aggressively by default, reveal detail on-demand”* ¹.

3. Physics Simulation Optimizations

The visualization currently uses a **force-directed physics simulation** (via d3-force in 3D) to position nodes. This is effective for small graphs, but with thousands of elements, it can become slow or chaotic. We suggest the following optimizations to the physics and layout engine:

- **Multi-Stage Layout:** Leverage the hierarchy to perform layout in stages. For example, at a high level, compute a coarse layout of clusters first, then layout nodes within each cluster locally. The current implementation already does a form of this for *“Systems view”*, grouping nodes by `node.module` and computing cluster centers separately ²². We can extend this idea:
- Position top-level clusters using a force simulation (treat each cluster as a big node repelling others). Use inter-cluster edge counts as weights to pull related clusters closer ²³.
- Within each cluster, either use a secondary force simulation or a simple grid/spiral layout to arrange the internal nodes around the cluster center ²⁴. Because clusters are laid out separately, the overall computation is partitioned, which can be more efficient than one giant simulation.
- Ensure that when clusters expand/collapse, the layout adjusts locally rather than restarting global layout from scratch. For instance, when revealing a cluster’s internals, **seed the child nodes’**

positions near the parent cluster's last position (with a bit of jitter) ²⁵. This way the newly added nodes “bloom” out of the parent smoothly, and the physics simulation only needs to minorly adjust, rather than causing a full graph shake-up. This mitigation addresses the risk of the graph “exploding and reforming” on every LOD change ²⁶.

- **Force Parameter Tuning:** Continue to leverage **degree-weighted forces** as done currently. The engine already increases repulsive force for high-degree nodes to give them more space ²⁷ and lengthens links that connect high-degree nodes ²⁸, which helps prevent dense hubs from clumping. These are good practices. We can further:
- **Enable Barnes-Hut optimizations** for the n-body (many-body) force if not already (d3-force has this built-in when using many-body force; ensure it's on to handle 2k nodes efficiently by approximating distant interactions).
- Adjust the **collision radius** to account for node size and degree ²⁹ – which is already done with log padding in current code. If overlap remains an issue in dense areas, slightly increase the collision radius or iterations ³⁰, though at cost of performance.
- The **layered “balloon” layout** (spherical shells by type) used in the 3D “Map” view is a clever way to seed the layout ³¹ ³². We should preserve this notion of layering by node type (so e.g. UI/front-end nodes start in one region, backend/data nodes in another shell). In fact, we might make it more explicit: consider constraining certain layers to specific **Z planes** in a semi-3D layout (e.g., UI layer nodes on one horizontal plane, backend on another, with connecting edges rising between planes) ³³. This would use a modified forceZ or custom constraints to group layers, making the 3D layout more interpretable (like a layer cake of architecture). However, this must be balanced with physics so it doesn't overly constrain related nodes from clustering.
- **Performance Tweaks:** For runtime performance, implement techniques to limit work:
 - **Dynamic Simulation Control:** The current viewer likely runs the force simulation continuously for an initial period or until stabilization. We can cap the simulation iterations or use an adaptive cooling so that after a certain point (or if the graph is static), the simulation stops and frees up CPU/GPU. Additionally, when the user is not interacting or when in an overview LOD, the physics could run at a lower intensity or even be paused (since high-level clusters might be relatively stable positions).
 - **Offload Computation:** If possible, move heavy computations off the main thread. The library `react-force-graph-3d` doesn't natively use web workers for physics (it runs in the main UI thread), but we could explore using a Web Worker to compute layout positions in the background for large graphs, then feed the positions back to the renderer. Another approach is precomputing an initial layout (or cluster positions) server-side or during the build and storing coordinates in the JSON – then the front-end can use those as initial positions to reduce time-to-first-render.
 - **Cache and Reuse Layouts:** Avoid recomputing layouts from scratch on every view change. For example, **cache the node coordinates for each cluster/LOD** once computed ³⁴. Then, if the user switches back to a higher level or revisits a cluster, you can reuse the last known positions instead of recalculating. This also provides visual stability (less jitter when toggling views).

In summary, the physics engine should become more **LOD-aware**. Run simpler, coarse simulations for high-level views, and localized simulations for local details. By doing so, we keep the system responsive and avoid unnecessary work on portions of the graph that the user isn't focusing on.

4. Rendering Performance and Techniques

Rendering 2k+ nodes and 5k edges is non-trivial, but using **WebGL** is the right choice – it can handle more objects than SVG/Canvas by leveraging the GPU. The project already uses a WebGL renderer (`react-force-graph-3d` with Three.js) and even provides an SVG fallback. Going forward, focus on **WebGL-first optimizations** ³⁵ ³⁴ :

- **Level of Detail Rendering:** Tie the rendering detail to the zoom level and performance mode. For example, at far zoom (overview), do not even attempt to draw individual small nodes or labels for each symbol. The code already throttles label rendering when moving and possibly hides labels on distant nodes – continue that approach with clear rules (e.g., **only render labels for cluster nodes at LOD0/1**; at LOD2+, render file/function labels *only* if they fit without overlap) ³⁶ . Implement a **label budget**: e.g., never draw more than N labels on screen to avoid GPU overload or text clutter ³⁶ . This might mean showing labels for the largest or most important nodes first, and omitting others until you zoom further. Additionally, ensure some form of **label collision avoidance or culling** so that labels don't overlap illegibly in dense areas ³⁶ .
- **Culling and Partitioning:** Utilize frustum culling or distance culling for nodes/edges. If the user zooms into a particular cluster, many other parts of the graph might be off-screen or far away – those could be omitted from rendering. A spatial index (like an octree or quadtree in 2D) can help quickly determine which elements are in view ³⁷ . The `react-force-graph` library might not do this by default for 3D, but a custom approach could hide objects outside the camera's view or beyond a certain distance threshold (especially if using an orthographic/top-down view mode).
- **LOD Switching Performance:** When swapping between LOD datasets, be careful to avoid expensive re-renders with thousands of DOM operations. Ideally, update the graph by *adding/removing groups of nodes* rather than destroying all and adding all anew. This is where using a canvas/WebGL library helps (since it can reuse buffers). Also, as noted, avoid recalculating layouts during view switches by caching positions ³⁴ .
- **Edge Rendering:** A major performance sink can be drawing 5k edges, especially if using heavy geometry or if each edge is an independent Line object. Consider batching edges:
 - Use **shader lines or instanced line rendering** so that all edges are drawn in a single GPU draw call (or a few) instead of thousands. Libraries like Three.js allow using `LineSegments` or custom shaders to draw many segments efficiently.
 - When edges are very numerous and dense, consider drawing them in a simplified way. One idea is to use **curved “tubes” or splines only at aggregated level** (Systems view) where edges are bundled (so only a few thick curves to draw). Another idea: in extreme cases, forego drawing all edges and use a **heatmap or matrix view** (outside of the 3D view) to show connectivity – though that might be beyond scope. More practically, toggling off certain edge types can instantly drop the count (e.g., turn off low-importance edges).
- **Visual Effects Simplification:** Make sure any visual embellishments are tuned for performance. For instance, if using particle animations for edges or fancy glow effects for nodes, those can hurt performance at scale. It might be wise to **disable expensive effects in “performance mode”** or

beyond a certain node count. The code's `enablePerformanceMode` and `reduceDetail` toggles should be leveraged to dial down detail when the graph is large ³⁸ ³⁹. This could include disabling dynamic shadows, reducing edge curvature detail, or simplifying node shapes.

- **Leverage Latest Tech:** Keep an eye on emerging tech like **WebGPU** for potentially massive graphs, but the current stack (Three.js + WebGL) is proven and can be optimized. Ensure Three.js is updated to latest for efficiency improvements. Also, verify that the **GPU memory usage** is controlled: use low-poly or point-sprites for nodes (likely already using sprites for labels ⁴⁰) and reuse materials to avoid overhead.

Overall, rendering performance will greatly benefit from **drawing fewer things** (hence the emphasis on LOD and filtering) and **batching** what is drawn as much as possible. By only rendering what's relevant at a given view and using WebGL instancing/batching, the app will remain fluid even as data scales up.

5. User Experience & Usability Improvements

Beyond raw performance, the tool's **usability** can be significantly improved with thoughtful UI/UX features. These will help users navigate and make sense of the graph:

- **View Modes & Presets:** Introduce distinct view modes that answer common questions, with one-click toggles ⁴¹:
- *Architecture Layers View:* Show the system as layered planes (UI → API → Domain → Data → Infra). This could leverage the semi-2D layout idea where each layer's nodes lie in a plane at different Z, giving a clear "layer cake" of the architecture ³³. Edges between layers then stand out as vertical or arched links connecting layers, which is easier to interpret in a static image.
- *Dependency Graph (Systems View):* Emphasize package/module dependencies (essentially the LOD0/1 view). This would highlight cross-boundary connections between major modules (like an import graph at a high level), effectively an **architecture dependency diagram**. The current "hierarchical (systems) view" with cluster boxes is along these lines; ensure it's easily accessible.
- *Execution Flow (Runtime call view):* Filter to only show edges that represent runtime calls or data flow (hide pure import relationships). If someone wants to trace how a request or function call propagates, this view is ideal ⁴². Potentially integrate with a feature to select an entry-point and automatically highlight a plausible call chain.
- *Hotspot or Complexity View:* Use node sizing or coloring to show "hot" areas (e.g., by commit churn or bug count). A preset could make node size proportional to LOC or complexity, and color by recent activity, giving a heatmap of the codebase health ⁴³. Important: ensure legends or tooltips explain these visuals.
- *Risk/Cycle View:* A mode to find *trouble spots* – e.g., highlight **circular dependencies** (could be detected by SCC algorithms in the graph data) or overly coupled components. Edges that are forbidden or problematic can be colored differently. This helps in architecture governance and is another perspective a preset can provide ⁴³.
- **Interactive Filtering & Search:** Provide controls to filter nodes/edges by various criteria ⁴⁴. Examples:

- **Edge type filters:** checkboxes or toggles to show/hide certain edge categories (imports, calls, tests, etc.) ⁴⁵. Users might temporarily hide all test relationships or all database access calls to reduce noise.
- **Node type filters:** e.g., only show code vs. documentation or only show application code vs. third-party libraries (if those are included in the graph). A toggle for “hide external/vendor nodes” would remove third-party library nodes if those were captured ⁴⁶.
- **Intra-module toggle:** a specific toggle to hide edges that stay within the same file or same module ⁴⁴. Those internal calls may not be useful in a high-level view and just clutter (imagine a class calling its own private method – not important for architecture view).
- **Search bar:** allow text search for a node by name. Given the scale, a user will want to locate a specific class or function quickly. The search can auto-zoom/focus on the found node, and maybe highlight its immediate connections.
- **Focus mode improvements:** The app already dims non-neighbor nodes when selecting a node ⁴⁷. Building on that, implement a “**radial focus**” limit – when a node is selected, limit the radius of how far out to show connected nodes, especially in a dense graph. For example, show the selected node, its direct neighbors, and maybe second-hop neighbors, but hide or strongly fade anything beyond that radius ⁴⁸. Also consider ranking those neighbors: if a node has 50 neighbors, perhaps show only the top 10 most significant ones by weight or type, and hide the rest until requested ⁴⁹. This keeps focus views readable.
- **Cluster Expand/Collapse Controls:** In addition to zoom-driven LOD, users should manually control cluster expansion:
 - Provide an “**expand**” icon on cluster nodes (e.g., a plus/minus or double-click to expand/collapse a group). This explicit control ensures a user can drill into a specific area even if they haven’t zoomed in deeply, and conversely collapse it to reduce detail.
 - Show a **breadcrumb or mini-map** of where the user is in the hierarchy. For instance, if they’ve expanded `Core → Services → AuthService`, display that path so they know the context. A mini-map could be as simple as a tree view sidebar or a small overview window highlighting the visible region in the context of the whole codebase graph.
- **Visual Styling and Clarity:** A few enhancements to make the visualization clearer:
 - Use consistent **color coding** for node types or layers. For example, all *UI layer* nodes (pages/components) could be one hue, *backend services* another, *database or models* another, etc. This, combined with the ARCHITECTURE_FLOW layering, helps users immediately distinguish parts of the system. A legend for colors/types would be useful.
 - Implement **summary glyphs or badges** on cluster nodes ⁵⁰ ⁵¹. Instead of a cluster node just being a generic circle, it could display mini indicators: e.g., a pie chart showing composition of that cluster (60% frontend, 40% backend code inside), or an icon indicating it has X number of files, or a heat color if that cluster contains a lot of churn or errors ⁵¹. These at-a-glance cues give meaning without needing to expand the cluster.
 - **Label cleanup:** The label cleaning logic in code (trimming file paths, etc. ⁵² ⁵³) is good – ensure it’s applied everywhere. Show shorter, human-readable names on nodes (no full paths unless on hover). For cluster nodes, use aggregated names (maybe common directory names). The viewer might

include an on-hover tooltip to show full details (full path, documentation string, etc.), keeping the canvas labels short.

- **Camera & Projection Options:** For usability in analyzing and exporting diagrams, consider adding **camera presets** and an **orthographic view** option ⁵⁴. For instance, a top-down orthographic projection can turn the 3D force layout into a 2D schematic that's easier to read (no perspective distortion, no overlapping due to depth) – useful for slides or documentation exports. The app could let the user toggle perspective vs. orthographic for clarity. Also, preset camera positions like “*top-down overview*”, “*side view*” (to emphasize layered planes), or “*isometric*” can help users find an optimal angle quickly ⁵⁴.

All these UX improvements aim to make the tool **more interactive and informative**, while preventing the user from getting lost in a sea of nodes. By allowing users to filter out noise, focus on areas of interest, and see the system at different abstraction levels, the codebase map becomes far more usable for real-world understanding and debugging.

Conclusion

By implementing the recommendations above – **richer graph metadata, hierarchical clustering, LOD-based rendering, physics tuning, and user-friendly controls** – the codebase map will transform from a sluggish hairball into a scalable “**map of the code**”. The guiding principle is to **show the right level of detail for the right context**: an overview that reveals system architecture, and drill-downs that expose implementation details only when needed. This multi-scale approach attacks the root cause of lag and clutter (excess edge density) by slicing the graph into meaningful layers and filtering out trivial details at higher levels ¹.

In practice, these changes will yield **immediate performance gains** (fewer elements rendered and simulated at once) and **significant usability improvements** (the visualization will highlight structure and insights rather than overwhelm the viewer). The codebase-map project already has a strong foundation and many of these ideas outlined in its plans; executing them step by step – starting with data schema updates and cluster logic, then moving to LOD rendering and UX features – will result in a powerful tool that scales to large codebases.

With an aggregated, layered graph, you'll be able to explore a 10k-node codebase fluidly, capturing high-level architecture on one screen and zooming into function-level logic on the next, all while maintaining clarity. These enhancements will make the codebase map not just a visualization, but an interactive atlas of the system's code – **performant, navigable, and insightful**.

Sources:

- Project “Visualization Improvements” Plan ¹ ⁵⁵ ¹⁸ ⁴⁸
- Layering & Clustering Design Doc ⁸ ¹³ ³⁷ ³⁶
- Graph Engine Overview (current state) ²² ²⁸ ⁴⁷
- Feedback on Layering Plan ²⁵ ¹⁷ (performance and implementation considerations)

1 2 3 5 6 10 11 12 18 20 21 33 42 48 49 54 55 **visualization-improvements.md**

https://github.com/miethe/codebase-map/blob/8ffe08dd434c27cb97a76f4a9f2ae34800fd0c6a/docs/project_plans/visualization-improvements.md

4 7 8 9 13 14 15 34 35 36 37 41 43 44 45 46 50 **visualization-layering-plan.md**

https://github.com/miethe/codebase-map/blob/8ffe08dd434c27cb97a76f4a9f2ae34800fd0c6a/docs/project_plans/visualization-layering-plan.md

16 17 25 26 51 **feedback-visualization-layering.md**

https://github.com/miethe/codebase-map/blob/8ffe08dd434c27cb97a76f4a9f2ae34800fd0c6a/docs/project_plans/feedback-visualization-layering.md

19 22 23 24 27 28 29 30 31 40 47 **webgl-engine.md**

<https://github.com/miethe/codebase-map/blob/8ffe08dd434c27cb97a76f4a9f2ae34800fd0c6a/docs/webgl-engine.md>

32 38 39 52 53 **GraphCanvasWebGL.tsx**

<https://github.com/miethe/codebase-map/blob/8ffe08dd434c27cb97a76f4a9f2ae34800fd0c6a/components/GraphCanvasWebGL.tsx>